Part 1 Technical Documentation - Oulu SDMO Group 23

- **Members:** Kaiquan Mah | Quoc Nguyen | Danish Sharjeel | Manh Nguyen | Kim Nguyen
- **Python:** 3.13, package manager `uv`, deps `cryptography>=50.0.1`, `mlkem>=0.0.3`
- **Tests:** 33 passed (was 27/33 baseline)
- This part 1 documentation consolidates our other documentations in the `documentation` folder which were focusing on separate parts of the system (01-06) + README + codebase. **So in this part 1 documentation, we cover the system requirements, architecture, design decisions, implementation, testing, operations, final evaluation of technical debt, references.**

Table of Contents

0. Acronyms
1. Summary
2. Requirements Mapping
3. Baseline Analysis (full from 01_)
4. Architecture (2-tier → 3-tier)
5. Design Decisions & LLM Interactions (full from 02_, 10 interactions + 3 fixes)
6. ML-KEM Integration (full from 03_)
7. Migration & Fallback Strategy (full from 04_)
8. Operations (full from 05_, CI/CD, Docker, logging, metrics, DB analytics, PQC observability)
9. Testing & Evaluation (33/33 tests)
10. Final Evaluation (full from 06_ strengths, weaknesses & tech debt, risks R1-R7)
11. References
12. Appendix: Files, LLM Prompts

---

0. Acronyms

- **PQC** = **Post-Quantum Cryptography**: crypto resistant to quantum computers
- **ML-KEM** = **Module-Lattice-based Key Encapsulation Mechanism** (NIST FIPS 203): PQC KEM
- **KEM** = **Key Encapsulation Mechanism**: one side encrypts random secret with public key, other decrypts
- **PSK** = **Pre-Shared Key**: same 16B key flashed into all devices at factory
- **AES-GCM** = **Advanced Encryption Standard - Galois/Counter Mode**: symmetric AEAD (confidentiality + 16B tag for integrity)
- **pk** = **public key** (ML-KEM encaps key 800B), **sk/dk** = **secret/decaps key** 1632B private key, **ct** = **ciphertext** 768B, **ss** = **shared secret** 32B (becomes AES-256 key), **B=Bytes**
- **TTL** = **Time To Live**, **OOM** = **Out Of Memory**, **LRU** = **Least Recently Used**
- **MITM** = **Man-In-The-Middle**, **DoS** = **Denial of Service**, **TLS** = **Transport Layer Security**
- **HSM** = **Hardware Security Module**, **KMS** = **Key Management System**, **DB** = **Database**, **ORM** = **Object-Relational Mapper** (we avoided)
- **E2E** = **End-to-End**, **EOF** = **End-Of-File** (close TCP to signal end), **MTU** = **Maximum Transmission Unit**
- **CI/CD** = **Continuous Integration/Continuous Deployment**, **SRAM** = **Static RAM**

---

1. Summary

Goal: Modernize small legacy edge-cloud system made up of a simulated legacy sensor to edge gateway to the cloud, from using traditional cryptography to using Post-Quantum Cryptograpcy (PQC), using LLM as an untrusted assistant, with humans evaluating LLM responses and choosing which to use or modify.

What we built:

- 2-tier baseline system (legacy → gateway, where 27/33 tests passed)
- 3-tier target system (legacy → gateway → cloud) with ML-KEM-512 hybrid implementation between gateway and cloud communications. 33/33 tests passed
- Gateway hardened
- Added schema validation for sensor data
- Replay cache
- Cloud Service (simulated; not a real cloud service like AWS/GCP): SQLite to store sensor data, health checks
- Fallback: If public key is missing or cloud service is unreachable (TCP 6000 refused/timeout), the gateway appends sensor data to gateway logs for business continuity (per CISA hybrid guidance)
- Operations: Dependency management via uv, pyproject.toml, Docker container for reproducible environment, CI/CD via GitHub actions

Changes made in part1 documentation v2 to address peer review feedback

- Feedback 1 to 5

```
- A brief section with a concise executive summary and some high-level diagrams may help also other types of stakeholders (like a project manager) to understand the whole process in a
- Some parts felt overly padded
- In technical report there should not be instructions on how to run the system those belong to README in the Github. Also 14 pages on the report seem excessive
- I'd maybe leave code specific stuff to the repo rather that put it here but that is my opinion
- Some parts are hard to read since there are so many code snippets in sentences. A lot of the content feels like it should be left into the repository
```

  - **We had summary section 1 in v1 of this documentation. We have simplified the summary further so only the major parts of what we build are above, for a non-technical audience**
  - Also, we had no idea how technical our peer reviewers will be. So our original idea was that (1) the professor has background in cryptography, and (2) the documentation might be the only document reviewed during peer review, so we intended to be as comprehensive as possible by including as much technical details as we could in v1 of our documentation. To be frank, Danish was the person with some cryptography background in the group, so for the rest of the group, it was also difficult to understand all the technical details (e.g. how the bytes are calculated for each part of the encryption/decrpytion/encapsulation/decapsulation method)
  - **So we have simplified section 4 in v2 of this document, to reduce the information overload (i.e. 'reduce padding')**
  - **Also, please refer to the baseline architecture diagram in section 3, and final architecture diagram in section 4**
- Feedback 6

```
- many chapters could have been summarized like tests being only one section not five
```

  - **We have kept baseline tests in section 3 baseline, as the baseline tests are different from the tests on the final system (which has a different architecture)**
  - **For the remaining test, we have moved from separate sections into only section 9 test**
- Feedback 7 to 9

```
- font size is too small
- Very small font size
- Text is really small
```

  - **Because GitHub's Markdown renderer does not support custom font sizes, we are unable to change the font size of this document we created in markdown format**

    - **If possible, please try to**
        - **Use a monitor display**
        - **You may wish to review the markdown version of this v2 documentation instead**: https://github.com/kai-oulu/oulu-sdmo-group23/blob/main/documentation/part1_documentation_v2_for_final_submission.md
        - **Zoom in/out** - whether you are reviewing the PDF or markdown version of this v2 document
- Feedback 10

```
- Work on R2 and R7
```

    - As we are finalising the documentation for submission, we will not be working on refining our build to cater for the risks discovered - R2 and R7, which will be part of our backlog

---

2. Project Requirements Mapping

Requirement	Status	Evidence (file:line)
AI-assisted development: generate components with LLM, record prompts/outputs/decisions	Done	`02_` logs 10 major LLM interactions (1 architecture, 2 protocol, 3 framing, 4 test plan, 5 test case generation, 6 gateway hardening, 7 ML-KEM, 8 cloud+DB+analytics, 9 forwarding+fallback, 10 Docker+CI)
Baseline analysis: understand initial architecture, run system, baseline tests/measurements	Done	`01_` with 2-tier diagram, measurements plaintext 95B, wire 123B, overhead 28B (12 nonce+16 tag), 27/33 tests initially, gap analysis
Critical evaluation & maintenance: test, review, correct, improve LLM artefacts	Done	Fixed gateway crash → 33/33, corrected `mlkem` API (`key_gen` not `generate`), fixed logging to file (was polluting stdout), fixed large packet test payload, fixed dual-wheel `mlkem` import (Rust vs pure Python), fixed missing `import os` in cloud_service
ML-KEM integration	Done	`src/pqc_utils.py` , `cloud_service.py` , `edge_gateway.py` , `file-based public key distribution via cloud_public_key.bin` , supports both Rust binding (rust.so 909KB) and pure Python (fastmath.so 80KB, empty **init**.py) python library wheels
Preserve non-PQC legacy sensor compatibility / migration & fallback strategy	Done	`04_` Phased migration approach
Operations: automate build/testing, deployment to `test` , health checks, logs, metrics	Done	`05_` + `.github/workflows/ci.yml` , `Dockerfile` , `docker-compose.yml` , sqlite DB, logging file-only (for logging if cloud service is offline), metrics, PQC observability
Final evaluation: strengths/weaknesses, technical debt, risks	Done	`06_` with acronym glossary, strengths, weaknesses, technical debt, risks mitigated/remaining
Simple DB at cloud side (SQLite) to store telemetry	Done	`telemetry.db` 1 table `telemetry(id, device_id, timestamp, temperature, humidity, status, received_at, source_ip, pqc_decaps_ms, raw_json)` , stdlib sqlite3, gitignored
Observability includes simple analytics on cloud side (avg temp, counts, PQC success rate)	Done	Cloud health 6001 returns `db_rows` , `analytics.avg_temperature` , `avg_humidity` , `last_10` , `pqc.decaps_ok` , `decaps_fail` , `success_rate` , `avg_decaps_ms`

Usage of AI:

- Must use AI to create code (Part1): Yes, all code in the `src` folder was generated via LLM prompts logged in 02_
- Must manually evaluate AI-generated code continuously: Yes, via pytest, log pollution fix, library fix (eg to cater for RUST which was introducted when we integrated with the mlkem library and GitHub actions), os import fix
- Must include prompts in technical documentation: Yes, in `02_` section 1 + appendix below
- Must report how/where AI used: Yes, accepted/modified/rejected matrix in `02_` section 1 + appendix below

---

3. Baseline Analysis (Full details from 01_baseline_analysis.md)

3.0 Summary

- **Initial architecture 2-tier only:** `[Legacy ATmega328P sim] --AES-128-GCM PSK 16B, TCP 5000, EOF--> [Edge Gateway]`
- **Measurements:** plaintext JSON ~95B, wire packet 12B nonce + 95B JSON + 16B tag = 123B, overhead 28B, 0B HTTP overhead (raw TCP+EOF)
- **Baseline tests:** 27 passed, 6 failed documenting risks R1-R3 (gateway crash on `InvalidTag` exit 1, no schema validation, no replay protection)
- **Device constraints:** ATmega328P 2KB SRAM / 32KB Flash, ML-KEM-512 sk 1632B + workspace 1000-1500B = ~3KB >2KB → stack overflow, so PQC must be offloaded to gateway (Linux/Pi)
- **Planned improvements done on 2026-10-10:** try/except boundary, REQUIRED_FIELDS schema, seen_nonces replay cache, now 33/33 pass

3.1 Initial Architecture (2-tier)

```
+-------------------------------------------------------------------------------------------+
|                                    SYSTEM ARCHITECTURE                                    |
+-------------------------------------------------------------------------------------------+

 [       Tier 1: Sensor      ]             [ Tier 2: Edge Gateway ]

  +---------------------------+             +---------------------+
  |   Simulated Legacy Device |             |    Edge Gateway     |
  |    (legacy_device.py)     |             |  (edge_gateway.py)  |
  |                           |             |                     |
  | • Profile: ATmega328P     |             | • Linux / Pi host   |
  | • 2 KB SRAM / 32 KB Flash | AES-128-GCM | • Decrypts AES-GCM  |
  | • Symmetric encryption    | ----------->| • Validates schema  |
  | • Raw TCP / Port 5000     |   (Link 1)  | • Encapsulates PQC  |
  | • EOF message framing     |             | • Forwards to cloud |
  +---------------------------+             +---------------------+
```

After implementing 2-tier, we added cloud as 3rd tier with ML-KEM.

```
[       Tier 1: Sensor      ]             [ Tier 2: Edge Gateway ]            [ Tier 3: Cloud Service ]

+---------------------------+             +---------------------+            +---------------------+
|   Simulated Legacy Device |             |    Edge Gateway     |            |    Cloud Service    |
|    (legacy_device.py)     |             |  (edge_gateway.py)  |            |  (cloud_service.py) |
|                           |             |                     |            |                     |
| • Profile: ATmega328P     |             | • Linux / Pi host   |            | • Ingestion API     |
| • 2 KB SRAM / 32 KB Flash | AES-128-GCM | • Decrypts AES-GCM  |  ML-KEM-512| • Decapsulates PQC  |
| • Symmetric encryption    | ----------->| • Validates schema  | ---------->| • Stores telemetry  |
| • Raw TCP / Port 5000     |   (Link 1)  | • Encapsulates PQC  |  (Link 2)  |                     |
| • EOF message framing     |             | • Forwards to cloud |            |                     |
+---------------------------+             +---------------------+            +---------------------+
```

3.2 Component Roles & Communication Links

Link 1: Sensor Node to Edge Gateway (Implemented Baseline)

- **Medium:** Bare TCP socket (port 5000) over local network
- **Security Scheme:** Symmetric AES-128-GCM authenticated encryption using 16-byte Pre-Shared Key (PSK)
- **Message Framing:** Single-shot connection closure (EOF). Sensor transmits packet and immediately closes socket writer, eliminating memory overhead of length-prefix decoding on 8-bit hardware.

Link 2: Edge Gateway to Cloud Service (Post-Quantum Channel - Planned → Implemented)

- **Medium:** TCP / TLS or secure HTTP API (we used TCP 6000 + EOF for simplicity)
- **Security Scheme:** ML-KEM-512 Post-Quantum Key Encapsulation Mechanism
- **Role:** Edge Gateway acts as security translator, upgrading from low-overhead symmetric encryption to post-quantum-resistant cryptography, before sending payload over public Internet to Cloud Service.

3.3 Temperature Sensor Device Specifications

- Device: Arduino Uno (Microchip ATmega328P 8-bit AVR microcontroller)
- Key Specs: 16 MHz CPU single-core, 32 KB Flash non-volatile program storage, 2 KB SRAM working memory
- It gives plain sensor telemetry (e.g., Temperature, Device ID, Timestamp)
- Lightweight Python script simulating payload and packet size, no hardware or complex emulators required
- **Memory limit:** ML-KEM-512 requires keys e.g., 1632 bytes secret key, while working memory might need 1000-1500 bytes for computations. Adds up to ~3KB, exceeds 2KB SRAM ceiling and causes stack overflow crashes.
- We will use simple lightweight key based encryptions (AES-128 or ChaCha20). Sensor cannot handle anything more complex.
- **Baseline Cryptographic Approach:**
  - Symmetric encryption via AES-128-GCM
  - 16-byte Pre-Shared Key (PSK) stored in protected Flash memory
  - Fresh 12-byte cryptographically secure random nonce prepended to each message to protect confidentiality (against eavesdropping)
  - Automated 16-byte GMAC authentication tag appended to ciphertext to ensure tamper detection (protects against tampering/bit-flipping, integrity/authenticity)
- Payload output simple:

```
{
  "device_id": "sensor-node-01",
  "timestamp": 1726478400,
  "temperature": 21.8,
  "humidity": 55.4,
  "status": "OK"
}
```

3.4 How to Run the System (Baseline 2-tier)

Prerequisites: Python 3.13+, `uv` package manager

Installation & Environment Setup:

```
uv sync
```

Running the System Simulation (2 active processes):

1. **Start Edge Gateway (Terminal 1):**

```
uv run python src/edge_gateway.py
```

Expected: `Edge Gateway listening for legacy devices on port 5000...`

2. **Trigger Legacy Sensor Node (Terminal 2):**

```
uv run python src/legacy_device.py
```

Behavior: Generates reading, encrypts payload, connects to port 5000, transmits packet, cleanly closes connection.

3. **Verify Reception (Terminal 1):** Gateway decrypts and prints authenticated telemetry:

```
Received from device: {"device_id": "sensor-node-01", "timestamp": 1726478400, "temperature": 21.8, "humidity": 55.4, "status": "OK"}
```

3.5 What Baseline Tests and Measurements Are There

5.1 Functional Unit Tests (`tests/unit/test_legacy_device_and_gateway.py`):

- Telemetry schema integrity: verifies mandatory keys (`device_id` , `timestamp` , `temperature` , `humidity` , `status`) and types
- Cryptographic round-trip interoperability: validates data encrypted by `legacy_device.encrypt_payload` successfully decrypts to identical JSON text in `edge_gateway.decrypt_payload`
- Packet layout verification: verifies 12-byte random nonce correctly placed at index `[0:12]`
- Edge-case payloads: tests empty JSON dictionaries `{}` , multi-byte UTF-8 Unicode (e.g., `ääö` , Chinese)

5.2 Cryptographic Security & Boundary Tests (`tests/unit/test_security_and_invalid_inputs.py`):

- Key mismatch rejection: decryption with incorrect 16-byte key fails with `InvalidTag`
- Tamper proofing (bit-flipping): flipping 1 bit in ciphertext, tag, or nonce raises `InvalidTag` and rejects payload
- Type & length validation: exceptions raised when passing invalid key lengths (15 bytes), non-byte types, non-serializable objects
- Truncation defense: packets shortened by 1, 8, or 16 bytes fail GMAC tag verification

5.3 TCP Protocol & Integration Tests (`tests/integration/test_gateway_tcp_flow.py`):

- End-to-end TCP transmission: real socket delivery from client to listening gateway on isolated loopback port
- TCP stream chunking & fragmentation: packet sent in fragmented segments (1 byte + 12 bytes + remainder) correctly accumulated and parsed across `conn.recv()` boundaries

5.4 Robustness & Acceptance Tests (`tests/integration/test_gateway_failure_and_robustness.py`):

- Gateway fault tolerance: gateway process does not crash when encountering malformed packets, truncated data, empty connections
- Replay attack rejection: identical replayed packets detected and discarded
- Schema conformance: rejects authenticated payloads missing required telemetry fields
- Buffer overflow handling: gateway dynamically handles payloads larger than initial 1024-byte buffer

5.5 Baseline Measurements:

- Wire protocol overhead: Plaintext JSON ~95B, total encrypted packet 12B nonce + 95B ciphertext + 16B GMAC tag = 123B, exact cryptographic overhead **28B** constant
- Network overhead 0 bytes: Raw TCP + EOF framing avoids HTTP headers (200-500B extra per request) and WebSocket handshakes, preserving sensor battery life

3.6 How to Run Baseline Tests and Measurements

Running Automated Test Suite:

```
uv run pytest -v
uv run pytest tests/unit/ -v
uv run pytest tests/integration/ -v
```

Baseline Test Results & Gap Analysis:

```
========================== short test summary info ===========================
FAILED test_gateway_remains_available_after_malformed_nonempty_packet
FAILED test_gateway_remains_available_after_connection_closes_mid_packet
FAILED test_gateway_ignores_empty_connection_and_processes_later_valid_packet (actually passed in baseline? but 6 failed total)
FAILED test_gateway_remains_available_after_two_packets_share_one_connection
FAILED test_gateway_rejects_authenticated_non_json_payload_and_processes_valid_packet
FAILED test_gateway_rejects_wrong_telemetry_schema_and_processes_valid_packet
FAILED test_gateway_rejects_replayed_packet_and_processes_new_valid_packet
======================== 6 failed, 27 passed in 6.46s ========================
```

Analysis of Current Baseline Limitations:

1. **Lack of Gateway Recovery Boundary (Crash on Bad Input):** `edge_gateway.py` does not wrap packet decryption in `try...except`. If invalid, malformed, truncated packet arrives, `aesgcm.decrypt()` raises `InvalidTag`, which crashes entire gateway process (exit status 1). Subsequent legitimate clients refused.
2. **Missing Application Schema Validation:** Gateway assumes any successfully decrypted UTF-8 string is valid telemetry. Does not validate required JSON fields before processing.
3. **Vulnerability to Replay Attacks:** Gateway does not track observed nonces or enforce incremental timestamps. Attacker intercepting authenticated packet can replay it, gateway will process as fresh.

Planned Improvements (done 2026-10-10):

- Implement exception handling boundary in `edge_gateway.py` to log bad packets and continue listening
- Add JSON schema validation on decrypted payload (`REQUIRED_FIELDS`)
- Add nonce/timestamp cache to detect and reject replayed packets (`seen_nonces=set()`)

Verifying Cryptographic Tampering Manually:

```
uv run python -c "
import legacy_device, edge_gateway
payload = legacy_device.generate_telemetry()
packet = bytearray(legacy_device.encrypt_payload(payload, legacy_device.PRE_SHARED_KEY))
packet[15] ^= 0xFF  # Corrupt one byte
try:
    edge_gateway.decrypt_payload(bytes(packet), legacy_device.PRE_SHARED_KEY)
except Exception as e:
    print('Tamper detected successfully:', type(e).__name__)
"
```

Measuring Packet Overhead:

```
uv run python -c "
import json, legacy_device
raw = json.dumps(legacy_device.generate_telemetry()).encode('utf-8')
enc = legacy_device.encrypt_payload(legacy_device.generate_telemetry(), legacy_device.PRE_SHARED_KEY)
print(f'Plaintext Size:  {len(raw)} bytes')
print(f'Wire Packet:     {len(enc)} bytes (Nonce: 12, Tag: 16, Overhead: 28 bytes)')
"
```

---

4. Architecture

4.1 Final Architecture Diagram (3-tier, with PQC, DB, analytics)

```
[Legacy Device] --AES-GCM PSK, TCP 5000, EOF--> [Edge Gateway: 5000 data + 5001 health]
                                                  | decrypt, validate schema, replay check (seen_nonces set)
                                                  | metrics: received/rejected/replayed/forwarded/fallback
                                                  | load cloud_public_key.bin (800B)
                                                  | if ok: pqc_utils.encrypt_for_cloud() -> TCP 6000
                                                  | else: fallback to gateway_fallback.log
                                                  v
                                            [Cloud Service: 6000 data + 6001 health/analytics]
                                                  | split ct(768)||nonce(12)||aes_ct
                                                  | decaps(dk,ct) -> ss 32B, AES-256-GCM decrypt
                                                  | INSERT INTO telemetry.db (SQLite)
                                                  | analytics: COUNT, AVG(temp/humidity), last 10, decaps_ok/fail, avg_decaps_ms, success_rate
```

4.2 Remaining architecture details

DB Schema (SQLite, simple, from 05_):

```
telemetry(
  id INTEGER PK AUTOINCREMENT,
  device_id TEXT,
  timestamp INTEGER,
  temperature REAL,
  humidity REAL,
  status TEXT,
  received_at INTEGER,
  source_ip TEXT,
  pqc_decaps_ms REAL,
  raw_json TEXT
)
```

- raw sqlite file `telemetry.db` gitignored, created at runtime
- Analytics queries: `COUNT, AVG, last 10`

For the remaining technical architecture details, please refer to section 4 of `part1_documentation_v1_for_peer_review.md`, such as on:

- How the nonce appears in the communication link between sensor to gateway, and gateway to cloud
- Why Offload PQC to Gateway
- How many threads
- How each of the code files work, with their major functions

---

5. Design Decisions & LLM Interactions (full details from 02_design_decisions_n_key_llm_interactions.md)

5.1 Significant Input Prompts and LLM Interactions (10 + 3 fixes)

Interaction 1: Baseline Architecture & Scaffolding

- **Context:** Implement lightweight sensor simulation ATmega328P communicating securely with Edge Gateway
- **Prompt:** "Write two Python scripts using cryptography library. One is legacy_device.py which encrypts sensor JSON telemetry using AES-GCM and a pre-shared key, connects to 127.0.0.1:5000 via TCP, sends the data, and disconnects. The second is edge_gateway.py which listens on port 5000, receives the encrypted bytes, decrypts the payload with the same key, and prints the sensor data."
- **LLM Output:** Proposed `legacy_device.py` generating JSON, `AESGCM(PRE_SHARED_KEY)`, `os.urandom(12)` nonce prepended, TCP client, and `edge_gateway.py` with socket server loop slicing `[:12]` nonce, decrypting remainder, decoding UTF-8
- **Human Review:** Basic AESGCM seemed correct. Issue: `Import "cryptography" could not be resolved`. Action: Configured `uv init`, `uv add cryptography`, `uv sync`, set IDE interpreter to `./.venv`

Interaction 2: Protocol Selection & Transport

- **Prompt:** "Should we use WebSockets or HTTP REST with Flask/FastAPI to stream the sensor data to the gateway?"
- **LLM Output:** Outlined benefits of WebSockets (persistent bidirectional framing) and HTTP (standard REST), noted microcontrollers often use simpler TCP or MQTT
- **Human Decision: Rejected WebSockets and HTTP:** Real ATmega328P 2KB SRAM / 32KB Flash, HTTP/1.1 client stack with verbose text headers (200-500B extra per request) or WebSocket with Sec-WebSocket handshakes and framing masks would exhaust 2KB RAM. **Accepted Bare TCP:** Raw TCP sockets closest to embedded serial link (UART/RS-485)

Interaction 3: Packet Framing & Message Boundaries

- **Prompt:** "How should the edge gateway know when the sensor has finished sending the encrypted message over the TCP socket?"
- **LLM Output:** Suggested (1) Length-prefixed framing (4-byte binary integer header), (2) Delimiter-based framing (newline or null-byte), (3) Socket closure (EOF)
- **Human Decision: Rejected Delimiters:** Ciphertext contains arbitrary binary bytes, any delimiter byte could appear randomly inside encrypted stream, causing premature truncation unless base64-encoded (33% bloat). **Accepted Socket Closure (EOF):** Closing socket after transmission (`sendall` + close) allows gateway to read until `recv()` returns empty `b""`, keeps microcontroller code minimal

Interaction 4: Codebase Analysis, Test Plan (Topic Proposal)

- **Prompt:** "I am new to this codebase. Please analyze src/legacy_device.py and src/edge_gateway.py in detail. Based on their architecture, propose a good test approach. Break it down into big topics and sub-topics covering unit, security, integration, and boundary validation. Also suggest how we should test because the gateway is supposed to run a continuous listening loop."
- **LLM Analysis:** Analyzed `legacy_device.py` (generates dict, encrypts via AESGCM with fresh 12B nonce prepended, TCP 5000, sendall, close) and `edge_gateway.py` (binds 127.0.0.1:5000 SO_REUSEADDR, loops accept, reads 1024-byte chunks until EOF, slices nonce, decrypts, prints). Proposed Big Topics:
  i. **Topic1 Core Functional Unit Testing:** telemetry schema, crypto round-trip, wire format 12B nonce at [0:12], edge cases empty dict, Unicode
  ii. **Topic2 Cryptographic Security & Malformed Input:** key validation wrong 16B key, invalid length, non-bytes types, AEAD tamper resistance bit-flipping in ciphertext/tag/nonce, wire truncation 1,8,16 bytes, serialization boundaries non-JSON serializable, non-UTF8
  iii. **Topic3 Network & TCP Stream Flow Integration:** live TCP delivery on ephemeral loopback ports, TCP fragmentation reassembly (split sendall chunks)
  iv. **Topic4 Gateway Robustness, Failure Modes & Edge Boundaries:** survivability on malformed packets, mid-stream disconnects, zero-byte connections, framing boundaries multiple packets one connection, payloads >1024B, application-layer validation non-JSON, missing schema keys, replay vulnerability
  v. **Topic5 Advanced Scenarios:** multi-client concurrency 50 parallel sensors, fuzz testing Hypothesis
  - Proposed `GatewayProcess` class wrapping `subprocess.Popen` with stdout daemon thread pushing into `queue.Queue` to avoid pipe deadlocks, dynamic ephemeral port reservation `socket.bind(("127.0.0.1",0))` to avoid `EADDRINUSE`
- **Human Decision:** Accepted Topics 1-4, dropped Topic5 (single-threaded gateway, adding fuzzing libs adds unnecessary scope)

Interaction 5: Test Case Generation, Review and Run

- **Prompt:** "Please proceed to generate code for topics 1 to 4 only. Topic 5 will make the project too complex and add unnecessary scope, so please avoid that. Also please provide the steps and commands to run the tests."
- **LLM Output:** Generated 5 test files for pytest, provided commands
- **Human Review:** Code review looked good, ran `pytest -v` → **27 passed, 6 failed** (all unit 16/16 and basic TCP flow 2/2 passed, 6 robustness failed in `test_gateway_failure_and_robustness.py`). Technical analysis: failures not due to test errors but real architectural limitations in `edge_gateway.py` (lacks try/except around decryption crashing process on invalid inputs, no JSON schema validation, no replay check). Decision: preserved failing tests to show limitation in baseline

Interaction 6: Gateway Hardening (Fixing 6 Failing Tests)

- **Prompt:** "My edge_gateway.py crashes when it receives malformed AES-GCM packets (InvalidTag) and it accepts any JSON without validating required fields device_id, timestamp, temperature, humidity, status, and it doesn't check for replayed nonces. How should I harden it to pass these tests: test_gateway_remains_available_after_malformed_nonempty_packet, test_gateway_remains_available_after_connection_closes_mid_packet, test_gateway_remains_available_after_two_packets_share_one_connection, test_gateway_rejects_authenticated_non_json_payload_and_processes_valid_packet, test_gateway_rejects_wrong_telemetry_schema_and_processes_valid_packet, test_gateway_rejects_replayed_packet_and_processes_new_valid_packet? Keep it simple, single-threaded, with logging and metrics."
- **LLM Output:** Suggested wrapping decrypt in try/except InvalidTag, ValueError, UnicodeDecodeError, JSONDecodeError, logging warning and continuing, REQUIRED_FIELDS set and `issubset` check, `seen_nonces=set()` for replay, check `if nonce in seen_nonces: reject`, add after successful validation, metrics dict and logging to file
- **Human Decision:** Accepted try/except boundary, REQUIRED_FIELDS validation, seen_nonces set, metrics counters, logging to file. Modified: LLM suggested logging to stderr - changed to file handler `gateway.log` because tests use `stderr=STDOUT` merged pipe, logging to stderr polluted `wait_for_line` (caused "Received ...Gateway health listening" concatenated line). Modified large packet test payload to include REQUIRED_FIELDS + samples to still test buffer accumulation. Verification: `pytest -v` → 33 passed, 0 failed, manual E2E with 2 sends works

Interaction 7: ML-KEM Integration (PQC Library and Hybrid Design)

- **Prompt:** "I need to integrate ML-KEM-512 into my edge-cloud system. I have mlkem==0.0.3 library. Show how to generate keypair, encapsulate, decapsulate, and build hybrid encryption where ML-KEM shared secret (32B) is used as AES-256-GCM key to encrypt telemetry JSON. Wire format should be ct(768)||nonce(12)||aes_ct. Keep it simple, file-based public key distribution."
- **LLM Output:** Initially suggested `MLKEM512.keygen()` and `ek.encaps()` which do not exist in `mlkem==0.0.3`, suggested hybrid pattern KEM+AES-GCM correct, file-based pk distribution
- **Human Decision:** Rejected `MLKEM512.keygen()` - tested via `python -c "import mlkem; help(mlkem)"` and REPL, found correct API is `ML_KEM(ParameterSet.ML_KEM_512).key_gen() -> ek, dk` and `encaps(ek) -> ss, ct` and `decaps(dk, ct) -> ss`. Accepted hybrid pattern, file-based distribution, wire format, constants PK_LEN=800, CT_LEN=768, SS_LEN=32. Modified: added timing metrics encaps_ms, decaps_ms for PQC observability, logging. Verification: round-trip via `python src/pqc_utils.py` → packet 845B, encaps 0.03ms, decaps 0.04ms, decrypt matches original

Interaction 8: Cloud Service + SQLite DB + Analytics

- **Prompt:** "Create a cloud_service.py that listens on TCP 6000 for PQC packets (ct 768 + nonce 12 + aes_ct), decapsulates using ML-KEM-512 private key, AES-GCM decrypts, stores in SQLite telemetry.db with table telemetry(id, device_id, timestamp, temperature, humidity, status, received_at, source_ip, pqc_decaps_ms, raw_json), and has health/analytics endpoint on 6001 returning JSON with db_rows, avg temp/humidity, last 10, decaps_ok/fail, success_rate, avg_decaps_ms. Keep it simple, stdlib only, no Flask."
- **LLM Output:** Suggested sqlite3 stdlib, http.server for health (no Flask), threading for health server, metrics dict with lock, check_same_thread=False, CREATE TABLE IF NOT EXISTS, INSERT with ?, analytics queries COUNT, AVG, last 10 ORDER BY
- **Human Decision:** Accepted SQLite stdlib, http.server (rejected Flask), threading, metrics with lock, analytics queries. Modified: logging to file handler `cloud.log` not stderr. Verification: cloud alone → telemetry.db 12K created, pk 800B file, curl 6001 health returns JSON with db_rows 0, analytics empty, pqc metrics. E2E with gateway: 2 sends → DB 2 rows, health db_rows 2, avg_temp 21.8, decaps_ok 2, success_rate 1.0

Interaction 9: Gateway Forwarding + Fallback + Health

- **Prompt:** "Extend edge_gateway.py to forward telemetry to cloud via PQC: try load cloud_public_key.bin, if missing or cloud unreachable, append JSON to gateway_fallback.log with timestamp and increment fallback_local metric. Use pqc_utils.encrypt_for_cloud and TCP send to 127.0.0.1:6000. Add health endpoint on 5001 returning JSON with received, rejected, replayed, forwarded_to_cloud, fallback_local, cloud_fail, replay_cache_size, uptime. Keep it simple, stdlib http.server, thread-safe metrics."
- **LLM Output:** Suggested `forward_to_cloud()` with try load pk, try socket create_connection timeout, fallback to file append, metrics with lock, health handler
- **Human Decision:** Accepted forward_to_cloud logic, fallback log format `<timestamp> <json>`, metrics `forwarded_to_cloud`, `fallback_local`, `cloud_fail`, health thread 5001. Modified: removed print for health listening to keep stdout clean for tests, added metrics_lock. Verification: fallback test stop cloud, send legacy → `gateway_fallback.log` has entry, health 5001 fallback_local=1. E2E with cloud up: 2 sends → DB 2 rows, gateway forwarded=2, cloud decaps_ok=2

Interaction 10: Docker + CI/CD + pyproject.toml/uv.lock/.python-version

- **Prompt:** "Generate GitHub Actions CI and Dockerfile and docker-compose for Python project using uv package manager, with pyproject.toml, uv.lock, .python-version 3.13. CI should run pytest -v, test PQC utils, check imports. Dockerfile should be python:3.13-slim, respect locked versions via uv sync --frozen, and have HEALTHCHECK. docker-compose should have cloud and gateway services with healthchecks and volume sharing for public key. Keep it simple."
- **LLM Output:** Initially suggested `pip install -r requirements.txt` - rejected, project uses pyproject.toml + uv, not requirements.txt. Suggested Flask for health - rejected, we use stdlib http.server. Suggested `astral-sh/setup-uv` action, `uv sync --frozen`, Dockerfile with `pip install uv && uv sync --frozen --no-install-project`, `ENV PATH .venv/bin`
- **Human Decision:** Rejected requirements.txt, Flask. Accepted `astral-sh/setup-uv@v4` with cache, `uv python install`, `uv sync --frozen`, `uv pip install pytest`, `uv run pytest -v`, Dockerfile with `COPY pyproject.toml uv.lock .python-version README.md`, `uv sync --frozen --no-install-project`, `ENV PATH .venv/bin`, HEALTHCHECK via socket, docker-compose with `depends_on healthy` and bind mounts. Modified: added `[dependency-groups] dev = ["pytest>=8.0"]` to pyproject.toml. Verification: `pytest -v` 33 passed, Dockerfile copies correct files, CI yml contains checks for .python-version, pyproject.toml, uv.lock

Additional Fixes (CI failures fixed by a human):

- **Fix 11: ModuleNotFoundError: No module named 'src'** - `conftest.py` only added `src/` to path, so `from src.pqc_utils import ...` fallback failed. Fixed by inserting both PROJECT_ROOT and src/ into sys.path, and only fallback when error contains `pqc_utils`, not masking dependency errors. Files: `tests/conftest.py:17-18`, `src/edge_gateway.py:22-35`, `src/cloud_service.py:20-35`
- **Fix 12: ImportError: cannot import name 'ML_KEM' from 'mlkem'** - mlkem 0.0.3 has 2 wheels: Rust (rust.so 909KB, `__init__.py` exports ML_KEM) and pure Python (fastmath.so 80KB, empty `__init__.py`). CI Python 3.13 got pure Python. Fixed by trying Rust first, fallback to pure Python `from mlkem.ml_kem import ML_KEM` + `from mlkem.parameter_set import ML_KEM_512`, flag `RUST_BINDING`. File: `src/pqc_utils.py:16-80`, tested Rust 33 passed encaps 0.06ms, pure Python 33 passed encaps 0.82ms
- **Fix 13: NameError: name 'os' is not defined** in `cloud_service.py` line 63 `CLOUD_HOST = os.getenv(...)` - missing `import os`. Fixed by adding `import os`. File: `src/cloud_service.py:11`, also synced `edge_gateway.py` to use env vars `CLOUD_HOST`, `CLOUD_PORT`, `FALLBACK_LOG` via `os.getenv`

5.2 Design Decisions Matrix (Accepted, Modified, Rejected)

Decision Area	Option Selected (Accepted)	Alternatives Evaluated	Status	Rationale
PQC Placement	Offload ML-KEM to Edge Gateway	Run ML-KEM-512 directly on the sensor	**Rejected**	ML-KEM-512 requires ~1632B secret keys + 1000-1500B working memory (~3KB total). Sensor has only 2KB SRAM, causing unavoidable stack overflow.
Local Link Security	AES-128-GCM with 16-byte PSK	ChaCha20-Poly1305, Plain AES-CBC	**Accepted**	AES-GCM provides AEAD, ensuring both confidentiality and tamper detection (GMAC tag).
Transport Layer	Bare TCP Socket (port 5000)	WebSockets, HTTP/REST, MQTT	**Rejected**	WebSockets and HTTP introduce 200-500B header overhead per message, exhausting sensor RAM and wasting battery.
Packet Framing	Single-shot Connection Closure (EOF)	4-byte Length-Prefix Header, Delimiter framing	**Accepted / Modified**	EOF framing simplifies microcontroller memory management. (Future: persistent connections with length headers may be considered).
Key Distribution	Pre-Shared Key (PSK) in ROM	Dynamic Key Exchange (DH/ECDH)	**Accepted**	Embedded sensors commonly store factory-flashed PSKs in write-protected Flash ROM to avoid compute-intensive asymmetric handshakes.
Nonce Generation	Fresh 12-byte random nonce per packet	Sequential counter, Static Nonce	**Accepted**	AES-GCM security collapses if nonces reused with same key. 96-bit random nonce ensures freshness without persistent EEPROM writes.
PQC Library	mlkem==0.0.3 (supports both Rust and pure Python wheels)	kyber-py, liboqs, cryptography	**Accepted**	Minimal, pip-installable, works with uv + Docker, dual-wheel fix added
PQC Params	ML-KEM-512 (800B pk, 768B ct)	768 (1184B pk) / 1024 (1568B)	**Accepted**	Lowest overhead for gateway, 128-bit security sufficient for short-lived telemetry
Hybrid	KEM+AES-256-GCM	Pure PQC encrypt	**Accepted**	ML-KEM is KEM, not encryption
DB	SQLite stdlib	JSONL, ORM, Postgres	**Accepted**	Stdlib, 1 file, enables analytics queries
Health	stdlib http.server	Flask/FastAPI	**Accepted**	No extra dep, simple
CI	astral-sh/setup-uv, uv sync --frozen --all-groups	pip install -r requirements.txt	**Accepted**	Project uses pyproject.toml + uv.lock + .python-version, PYTHONPATH fix
Docker	python:3.13-slim + uv sync --frozen	pip install directly	**Accepted**	Respects locked versions, reproducible

5.3 Current System Limitations (from 02_)

- **Absence of Gateway Recovery Boundary (Crash on Bad Input):** If adversary transmits malformed bytes, truncated packet, or data encrypted with wrong key, `aesgcm.decrypt()` raises `InvalidTag`. In baseline `edge_gateway.py`, unhandled exception crashes entire gateway process (exit status 1). Fixed in final version with try/except.
- **Lack of Telemetry Application Schema Validation:** Gateway decrypts payload and blindly logs `Received from device: {raw_data}` as long as bytes form valid UTF-8. Does not validate required JSON fields. Fixed with REQUIRED_FIELDS.
- **Vulnerability to Replay Attacks:** Gateway processes any packet with valid GMAC tag. Eavesdropper who records valid transmission can re-transmit exact same byte sequence, gateway will accept as legitimate. Fixed with seen_nonces set, but no TTL/persistence remains as tech debt.
- **Single-Client Synchronous Blocking Loop:** `edge_gateway.py` executes single-threaded synchronous loop `accept()` + blocking `recv()`. While gateway reads/decrypts from one sensor, no other sensor can connect. If one client stalls, gateway hangs for all nodes. Documented as scalability tech debt.
- **TCP Handshake Overhead:** Using connection closure (EOF) as message boundary requires opening/closing TCP socket for each reading. Every reading requires full TCP 3-way handshake SYN→SYN-ACK→ACK and teardown FIN→ACK, introducing latency. Trade-off for simplicity.

5.4 Remaining Project Risks and Mitigation Roadmap (consolidated R1-R7, full from 06_ - authoritative risks table, also repeated in Section 10.4)

Note: 02_design_decisions_n_key_llm_interactions.md originally listed R1-R5 (baseline risks). After implementing PQC (Link2), SQLite DB, and file-based pk distribution, we added R6 (pk MITM) and R7 (DB backup) in 06_final_evaluation.md. The same table appears in Section 10.4 Final Evaluation.

#	Risk	Severity	Status	Remaining (what still needs fixing)
R1	Gateway DoS (Denial of Service) via malformed input (crash on InvalidTag)	High	**Mitigated:** try/except + schema + replay, now 33/33 pass, stays alive	Large payload could still cause high CPU when decrypting, should add rate limiting
R2	Replay attacks (stale telemetry reinjected)	Medium	**Mitigated:** seen_nonces set rejects replayed nonce (if nonce already seen, drop)	No TTL (Time To Live), restart clears cache → replay possible after restart. Should add timestamp window: only accept packets where `now - timestamp < 5 min`, and keep nonces only for that window
R3	PSK (Pre-Shared Key) extraction (physical tampering)	Medium	**Not mitigated:** global PSK still	Per-device keys, protected Flash

#	Risk	Severity	Status	Remaining (what still needs fixing)
R4	PQC performance & memory overhead	Medium	**Mitigated/measured:** encaps 0.03ms Rust / 0.82ms pure Python, decaps 0.04ms / 1.46ms negligible for gateway (Linux/Pi)	800B pk (public key) + 768B ct (ciphertext) overhead matters for constrained radio. **Constrained radio** = low-power wireless like LoRaWAN, Zigbee, BLE, NB-IoT with small MTU (e.g., 51-250B max), low bandwidth, battery-powered. Sending 1568B extra each message is heavy for them (needs fragmentation, more battery). In our simulated TCP/Ethernet setup it's fine, but for real battery sensor with LoRa it would be problem.
R5	Scalability & concurrent starvation	Low	**Not mitigated:** single-threaded data plane (2 threads total, 1 for data)	asyncio or thread pool future
R6	File-based pk (public key) MITM (Man-In-The-Middle)	Medium	**Not mitigated:** pk file no auth (authentication)	Sign pk or use secure distribution (cert, TLS pinning, KMS)
R7	DB (Database) no backup	Low	**Not mitigated:** single file `telemetry.db`	Periodic backup

Overall: High-severity R1 fixed, medium R2 partially fixed, others documented as technical debt. Full details also shown in Section 10.4.

---

6. ML-KEM Integration (full details from 03_mlkem_integration.md)

6.1 Objective

Integrate post-quantum key encapsulation (ML-KEM) into at least one communication path (Link 2) using an existing library, preserving legacy compatibility for non-PQC in Link 1 (ATmega328P).

6.2 Library Choice

Chosen library: `mlkem==0.0.3`

Alternatives evaluated:

- `kyber-py` (pure Python, suggested in course Q&A): heavier, slower, more code
- `liboqs` Python wrapper: requires native liboqs install, complex for Docker
- `cryptography` does not yet include ML-KEM in stable release

Decision: Accepted `mlkem`

- Minimal, pip-installable, no native deps beyond Rust compiled wheel (or fastmath for pure Python)
- API validated via REPL: `ML_KEM(ParameterSet.ML_KEM_512).key_gen() -> ek 800B, dk 1632B`, `encaps(ek) -> ss 32B, ct 768B`, `decaps(dk, ct) -> ss`
- Works with `uv sync --frozen` and `python:3.13-slim` Docker
- Documented as tech debt: file-based public key distribution (simple for 4h), future should use certificate or KMS
- **Dual-wheel fix:** Supports both Rust binding (rust.so 909KB, `__init__.py` exports ML_KEM) and pure Python (fastmath.so 80KB, empty `__init__.py`) - we handle both via `RUST_BINDING` flag, trying Rust first then pure Python fallback

Why ML-KEM-512 (not 768/1024):

Param set	Security	Public key	Ciphertext	Shared secret	Use case
ML-KEM-512	~AES-128 (Category 1)	800B	768B	32B	Lightest, constrained gateway
ML-KEM-768	~AES-192 (Category 3)	1184B	1088B	32B	NIST recommended default
ML-KEM-1024	~AES-256 (Category 5)	1568B	1568B	32B	Highest

- ML-KEM-512 has lowest wire overhead (800+768=1568B vs 2272B for 768) - matters for gateway forwarding many sensors
- Still 128-bit classical security, sufficient for telemetry with short secrecy lifetime (minutes/hours, not decades)
- Cloud stores telemetry short-term, not long-term archival
- Documented: if long-term confidentiality needed (e.g., medical), should use ML-KEM-768 or 1024

6.3 Hybrid Encryption Design (Link 2)

Why hybrid: ML-KEM is KEM, not encryption. It establishes 32B shared secret, then we use symmetric AES-256-GCM for actual data.

Wire format:

```
packet = ct (768B) || nonce (12B) || aes_ciphertext + tag (variable, ~65B for small JSON)
Total ~845B for typical telemetry {"device_id":..., "temperature":21.8,...}
```

Flow:

1. Cloud on start: `ek, dk = ML_KEM(ML_KEM_512).key_gen()`, save `ek` to `cloud_public_key.bin` (800B), `CLOUD_PK_PATH` env var (default `./cloud_public_key.bin`, Docker `/app/shared/cloud_public_key.bin` via volume `pqc-shared`)
2. Gateway after validating legacy AES-GCM payload:
   - Load `cloud_public_key.bin` (if missing → fallback to `gateway_fallback.log` + `fallback_local++`)
   - `ss, ct = mlkem.encaps(ek)` - encaps time logged (~0.03ms Rust / 0.82ms pure Python)
   - `nonce = os.urandom(12)`, `aes_ct = AESGCM(ss).encrypt(nonce, telemetry_json_bytes, None)`
   - Packet = ct + nonce + aes_ct, send via TCP to cloud:6000 (`CLOUD_HOST` env var, default 127.0.0.1 local, `cloud:6000` in Docker), EOF framing via `shutdown(SHUT_WR)`
3. Cloud: recv until EOF, split ct/nonce/aes_ct, `ss = mlkem.decaps(dk, ct)` (~0.04ms Rust / 1.46ms pure Python), `AESGCM(ss).decrypt(nonce, aes_ct)`, JSON parse, insert into SQLite

PQC observability logged:

- `encaps_ms`, `decaps_ms`, `ct_len=768`, `total_len`, `ss_len=32`, `success_rate = decaps_ok/(ok+fail)`, `rust_binding` flag
- Exposed via health endpoints 5001 (gateway) and 6001 (cloud)

6.4 Code Snippets (from 03_)

Keygen (cloud_service.py, Rust version):

```
from mlkem import ML_KEM, ParameterSet
mlkem_obj = ML_KEM(ParameterSet.ML_KEM_512)
ek, dk = mlkem_obj.key_gen()  # ek is EncapsKey, dk is DecapsKey
ek_bytes = ek.to_bytes()  # 800B
dk_bytes = dk.to_bytes()  # 1632B
Path("cloud_public_key.bin").write_bytes(ek_bytes)
```

Keygen (pure Python version):

```
from mlkem.ml_kem import ML_KEM
from mlkem.parameter_set import ML_KEM_512
mlkem_obj = ML_KEM(ML_KEM_512)
ek_bytes, dk_bytes = mlkem_obj.key_gen()  # bytes directly
```

Encrypt (pqc_utils.py):

```
if RUST_BINDING:
    ek = EncapsKey.from_bytes(ek_bytes)
    ss, ct = mlkem_obj.encaps(ek)  # ss 32B, ct 768B
else:
    ss, ct = mlkem_obj.encaps(ek_bytes)
nonce = os.urandom(12)
aes_ct = AESGCM(ss).encrypt(nonce, plaintext, None)
packet = ct + nonce + aes_ct
```

Decrypt (pqc_utils.py):

```
ct = packet[:768]
nonce = packet[768:780]
aes_ct = packet[780:]
if RUST_BINDING:
    dk = DecapsKey.from_bytes(dk_bytes)
    ss = mlkem_obj.decaps(dk, ct)
else:
    ss = mlkem_obj.decaps(dk_bytes, ct)
plaintext = AESGCM(ss).decrypt(nonce, aes_ct, None)
```

6.5 Security Properties

- **Confidentiality:** ML-KEM secure against quantum (Module-LWE hardness), AES-256-GCM secure
- **Integrity:** AES-GCM tag verifies tampering on Link2, same as Link1
- **No forward secrecy for PSK Link1** - documented as weakness (global PSK)
- **PQC Link2 has no forward secrecy either if dk leaked** - should rotate keys periodically (future)
- **Replay protection:** Gateway tracks seen nonces for Link1, cloud relies on gateway filtering + timestamp check (future: add nonce cache in cloud too)

6.6 Remaining Limitations

- File-based public key distribution (no cert, no rotation) - documented as Phase1, future should use KMS or mutual TLS with PQC certs
- No key rotation, no TTL on seen_nonces (in-memory set grows)
- ML-KEM-512 only, not 768 (trade-off for simplicity)
- Pure Python version slower (0.82ms vs 0.03ms) but still negligible for gateway

6.7 References

- NIST FIPS 203 ML-KEM standardised in August 2024 - https://www.encryptionconsulting.com/education-center/ml-kem-fips-203/
- ML-KEM sizes: 800B pk (public key / encapsulation key), 768B ct (ciphertext) - https://vucense.com/privacy-sovereignty/post-quantum-cryptography/ml-kem-fips-203-implementation-guide-2026/

---

7. Migration & Fallback Strategy (full details from 04_migration_fallback_strategy.md)

7.1 Legacy Constraint: Why Arduino Can't Do PQC

- Device profile: ATmega328P, 16 MHz, 32 KB Flash, **2 KB SRAM**
- ML-KEM-512 requirements: secret key 1632B + working memory 1000-1500B for polynomial operations = ~3KB total
- Result: **Stack overflow, crash** - physically impossible to run ML-KEM on 2KB SRAM
- Measurement from baseline: ML-KEM-512 secret key alone exceeds 80% of SRAM
- Conclusion: Legacy device must stay on lightweight symmetric crypto (AES-128-GCM, 16B key, 12B nonce, 28B overhead)
- Same as real-world IoT: many low-cost sensors cannot be upgraded to PQC without hardware replacement [1][2]

7.2 Phased Migration Strategy (Org-friendly, C-suite view)

Phase 0: Legacy Only (Baseline, initial repo)

- Link1: Legacy → Gateway: AES-128-GCM PSK, TCP 5000, EOF framing
- Link2: Gateway → Cloud: not implemented or plaintext
- Risk: harvest-now-decrypt-later on Link2 if internet-facing

Phase 1: Hybrid Offload (Current Implementation, 4h scope)

- Link1: **Unchanged** - legacy devices keep AES-128-GCM PSK, no firmware update needed
- Gateway: **Security translator** - decrypts AES-GCM, validates schema, checks replay, then encapsulates with ML-KEM-512 and AES-256-GCM to cloud
- Link2: **PQC-protected** - `ct(768)||nonce(12)||aes_ct`, cloud decaps and stores in SQLite
- Public key distribution: **File-based** - cloud writes `cloud_public_key.bin` (800B) on start, gateway reads file. Simple for simulation, documented as tech debt.
- Benefits: No legacy disruption, PQC on internet-facing link, fallback possible
- Trade-off: Gateway is single point of trust, must be hardened (done: try/except, schema, replay)

Phase 2: Future PQC-Capable Devices (Beyond 4h, roadmap)

- New devices with >8KB SRAM (e.g., ESP32, Raspberry Pi Pico) could run ML-KEM-512 directly
- Link1 could become hybrid: AES-GCM + ML-KEM, or pure PQC
- Gateway would support both old PSK and new PQC devices (negotiation)
- Cloud would support both ML-KEM-512 and 768 (for long-term confidentiality)
- Requires hardware replacement for old fleet - cost/time (10-20 years for large orgs per CISA)

7.3 Fallback Strategy (Business Continuity)

Goal: System stays available even if PQC fails, cloud down, or keys missing - critical for org.

Failure Mode	Detection	Fallback Action	Observability
Cloud public key file missing (`cloud_public_key.bin` not found)	`FileNotFoundError` on load	Gateway logs warning, appends telemetry JSON to `gateway_fallback.log` with timestamp, increments `fallback_local` metric	Health 5001 `fallback_local++`, log `cloud_public_key not available`
Cloud unreachable (TCP 6000 connection refused, timeout)	`ConnectionRefusedError`, `OSError`, `socket.timeout`	Same as above: fallback log + `cloud_fail++` + `fallback_local++`	Health 5001 `cloud_fail`, `fallback_local`, log `Cloud unreachable`
PQC decaps fail in cloud (corrupted ct, wrong key)	Exception in `decrypt_from_gateway`	Cloud logs warning, increments `decaps_fail`, does NOT crash, continues listening	Health 6001 `decaps_fail`, `success_rate`, log `Rejected PQC packet`
PQC encaps fail in gateway	Exception in `encrypt_for_cloud`	Fallback log + `fallback_local`	Health 5001, log `Unexpected error in forward_to_cloud`
Legacy malformed packet (InvalidTag, truncated, replay)	`InvalidTag`, `ValueError`, `seen_nonces` check	Gateway logs warning, increments `rejected` / `replayed`, continues listening (no crash)	Health 5001 `rejected`, `replayed`, logs, 33/33 tests pass

Store-and-forward: `gateway_fallback.log` format: `<unix_timestamp> <json>`, e.g., `1726478400 {"device_id":...}`. Operator can later replay to cloud manually or via script. Minimal persistence.

Why fallback matters for org: Per CISA/EU guidance, hybrid deployment with fallback ensures legacy devices keep working during PQC migration, avoiding downtime. If PQC implementation has bug, system degrades gracefully to local log, not total outage.

7.4 Legacy Compatibility Preservation

- **Link1 unchanged:** Same PSK `legacy_secret_16`, same 12B nonce prepended, same EOF framing - legacy device code not modified after baseline
- **No firmware update required** for ATmega328P - justified by SRAM analysis
- **Gateway per-device subkeys not implemented** (global PSK) - documented as tech debt, future should have per-device keys in protected Flash
- **Backward compatibility:** New gateway still accepts old legacy payloads (with required fields), old gateway would reject PQC packets (expected)

7.5 Remaining Risks

- File-based pk distribution has no authentication - attacker could replace `cloud_public_key.bin` with own key (mitigation future: sign pk file or use cert)
- `seen_nonces` in-memory set grows forever, no TTL - could OOM with many devices (mitigation: add TTL or LRU cache)
- Global PSK - if one device extracted, all compromised (mitigation: per-device subkeys)
- No key rotation for ML-KEM - dk stays in memory (mitigation: rotate daily, persist securely)

7.6 References

- EU roadmap, hybrid for legacy, hardware replacement may be needed - https://www.helpnetsecurity.com/2025/07/10/david-warburton-f5-labs-eu-pqc-roadmap/
- legacy constraints, computational overhead - https://eureka.patsnap.com/report-how-to-adapt-post-quantum-standards-to-existing-legacy-systems
- CISA PQC initiative, phased migration - https://www.cisa.gov/topics/risk-management/quantum

---

8. Operations (full details from 05_operations.md)

8.1 How to Run (Local, 3 Terminals)

Prerequisites: Python 3.13, `uv` package manager (or pip), `cryptography`, `mlkem`

Setup (once):

```
uv sync --frozen --all-groups
# or pip install cryptography mlkem pytest
```

Terminal 1: Cloud Service (data 6000 + health 6001)

```
uv run python src/cloud_service.py
# Expected:
# Cloud Service listening for gateway on port 6000...
# Cloud health/analytics listening on 6001...
# Generates cloud_public_key.bin (800B) and telemetry.db (SQLite)
```

Terminal 2: Edge Gateway (legacy 5000 + health 5001)

```
uv run python src/edge_gateway.py
# Expected:
# Edge Gateway listening for legacy devices on port 5000...
# Reads cloud_public_key.bin if exists, otherwise fallback
```

Terminal 3: Legacy Device (ephemeral)

```
uv run python src/legacy_device.py
# Sends one telemetry reading, closes
```

Verify:

- Gateway Terminal 2: `Received from device: {"device_id":...}`
- Cloud Terminal 1: `Cloud received from sensor-node-01: {...}` + `Cloud stored...`
- DB: `sqlite3 telemetry.db "SELECT * FROM telemetry;"`
- Health:
  - `curl http://127.0.0.1:5001/health` → gateway metrics `received, forwarded_to_cloud, fallback_local, replayed`
  - `curl http://127.0.0.1:6001/health` → cloud `db_rows, analytics avg_temp, pqc decaps_ok, success_rate, avg_decaps_ms`

Fallback test:

- Stop cloud (Ctrl+C in Terminal1), keep gateway running, send legacy again
- Check `gateway_fallback.log` → has `<timestamp> <json>`
- Health 5001 shows `fallback_local++`

8.2 Docker Deployment (Entire pipeline is simulated, just code)

Build:

```
docker build -t edge-gateway:test .
docker build -t cloud-service:test .
```

Run via docker-compose (local simulation):

```
docker-compose up --build
# cloud: 6000:6000, 6001:6001, volumes for pk sharing
# gateway: 5000:5000, 5001:5001, depends_on cloud healthy
```

Legacy device via Docker (ephemeral):

```
docker run --network host python:3.13-slim python src/legacy_device.py
# Or locally: python src/legacy_device.py
```

Dockerfile respects pyproject.toml + uv.lock + .python-version:

- `FROM python:3.13-slim` matches `.python-version` 3.13
- `COPY pyproject.toml uv.lock .python-version README.md`
- `RUN pip install uv && uv sync --frozen --no-install-project` - uses locked versions from uv.lock
- `ENV PATH="/app/.venv/bin:$PATH"`
- `HEALTHCHECK` tries 5000 or 6000 via socket
- `Dockerfile` implicit convention: `docker-compose.yml` `build: .` implicitly reads `./Dockerfile` (explicit `build: {context: ., dockerfile: Dockerfile}`), `ci.yml` `docker build -t ... .` implicitly `docker build -f Dockerfile -t ... .` - both read same Dockerfile without mentioning name

8.3 CI/CD Pipeline (full from 05_ + fixes)

File: `.github/workflows/ci.yml`

Triggers: push to `main` or `arena/*`, PRs to `main`

Jobs:

1. **test:**
   - `actions/checkout@v4`
   - `astral-sh/setup-uv@v4` with cache - reads `.python-version` (3.13) automatically
   - `uv python install && uv python pin 3.13`
   - `uv sync --frozen --all-groups` - respects `pyproject.toml` + `uv.lock` + dev group `pytest`
   - `uv pip install pytest` fallback
   - `uv run python -c "import cryptography; import mlkem; print('deps ok')"` - verifies deps
   - `env: PYTHONPATH: ${{ github.workspace }}` + `uv run pytest -v` → 33 tests (was 27/33 baseline, now 33/33 after hardening)
   - `uv run python src/pqc_utils.py` → round-trip test packet 845B, Round-trip OK, `rust_binding` flag
   - `uv run python -c "import src.pqc_utils; import src.edge_gateway; import src.cloud_service; print('imports ok')"` - ensures 3-tier imports work
   - `uv run python -c "import sys; sys.path.insert(0,'src'); import edge_gateway; import pqc_utils; print('top-level imports ok')"` - tests top-level imports used by conftest
   - Verify `pyproject.toml`, `uv.lock`, `.python-version` exist
2. **docker-build:** `needs: test`, builds both images `docker build -t edge-gateway:test .` and `docker build -t cloud-service:test .` to verify Dockerfile respects lock

Why CI matters: Ensures gateway hardening (33/33) doesn't regress, PQC utils round-trip works, imports ok for both Rust and pure Python wheels, Docker builds are reproducible.

Runtime table (linkage of files):

- `pyproject.toml+uv.lock+.python-version` used local+CI+Docker
- `Dockerfile` used local Docker + CI docker-build
- `docker-compose.yml` local only (CI does not run `compose up`, only `docker build` verification)
- `ci.yml` GitHub Actions only (push to main/arena/* PRs, `test` first then docker-build needs `test` to complete)

8.4 Monitoring, Logging, PQC Observability (full)

Logging:

- Gateway: `gateway.log` file handler, not stderr to keep tests clean, INFO level, timestamp, e.g., `2026-10-10 INFO Valid telemetry from sensor-node-01 temp=21.8`
- Cloud: `cloud.log` similarly
- PQC: `pqc.log` for encaps/decaps timings, `rust_binding` flag
- All logs use `logging.basicConfig` with file handler, minimal deps, gitignored `*.log`
- **Why file-only?** Tests in `conftest.py` use `GatewayProcess` which does `subprocess.Popen(..., stdout=PIPE, stderr=STDOUT, text=True)` merging stdout+stderr and reads line-by-line via `queue.Queue`, `wait_for_line` looks for `Received from device:`. If we log to stderr (default), logs go into same pipe as stdout and cause concatenated line `Received...2026-10-10 INFO Gateway health listening...` → test failure. Fixed by logging to file only.

Metrics (in-memory counters, thread-safe with lock):

- Gateway `metrics`: `received, rejected, replayed, invalid_json, invalid_schema, empty, forwarded_to_cloud, fallback_local, cloud_fail`, `replay_cache_size=len(seen_nonces)`, `uptime_s`
- Cloud `metrics`: `total_received, decaps_ok, decaps_fail, json_fail, db_rows, avg_decaps_ms, total_decaps_ms, uptime_start`

Health endpoints:

- Gateway 5001: `GET /health` → JSON with metrics, replay_cache_size, uptime
- Cloud 6001: `GET /health` or `/analytics` → JSON with `db_rows, total_received, analytics {total_count, avg_temperature, avg_humidity, max_timestamp, last_10}, pqc {decaps_ok, decaps_fail, json_fail, success_rate, avg_decaps_ms}, uptime_s`

PQC observability (post-quantum-related):

- Encaps time: `encaps_ms` logged in `pqc_utils.encrypt_for_cloud` (~0.03ms Rust / 0.82ms pure Python)
- Decaps time: `decaps_ms` logged in `decrypt_from_gateway` (~0.04ms Rust / 1.46ms pure Python)
- Sizes: `ct_len=768`, `pk_len=800`, `ss_len=32`, `total_len` ~845B for small JSON
- Success rate: `success_rate = decaps_ok/(ok+fail)` exposed via health
- Fallback rate: `fallback_local` and `cloud_fail` show PQC distribution failures
- Rust binding flag: `rust_binding` in metrics shows which wheel used

Analytics as observability (simple DB):

- SQLite `telemetry.db` with table `telemetry(id, device_id, timestamp, temperature, humidity, status, received_at, source_ip, pqc_decaps_ms, raw_json)`
- Queries: `COUNT`, `AVG(temperature)`, `AVG(humidity)`, `last 10 ORDER BY received_at DESC`
- Useful for managers: see how many messages protected by PQC vs fallback, avg temp, recent telemetry

8.5 Remaining Ops Gaps

- No Prometheus/Grafana (JSON health only, documented as simple)
- No log rotation (logs grow forever, should add `RotatingFileHandler`)
- No alerting (future: alert if `success_rate < 0.9` or `fallback_local` high)
- File-based pk sharing via bind mount not secure for production (should use secret manager/KMS)
- DB single file no backup, no replication

---

9. Testing & Evaluation

9.1 Pytest 33 Tests (from 01_ + 02_)

Breakdown:

- **5 unit legacy/gateway (`test_legacy_device_and_gateway.py`):** telemetry schema, crypto round-trip, wire format nonce at [0:12], empty dict, Unicode
- **11 security/invalid (`test_security_and_invalid_inputs.py`):** key mismatch rejection InvalidTag, tamper proofing bit-flipping in ciphertext/tag/nonce, type & length validation invalid key length 15B, non-bytes types, non-serializable objects, truncation defense 1,8,16 bytes, non-UTF8
- **2 TCP flow (`test_gateway_tcp_flow.py`):** end-to-end live TCP delivery on ephemeral loopback ports, TCP fragmentation reassembly (split sendall)
- **10 robustness (`test_gateway_failure_and_robustness.py`):** gateway survivability on malformed packets, mid-stream disconnects, zero-byte connections, framing boundaries multiple packets one connection, payloads >1024B, application-layer validation non-JSON, missing schema, replay attack
- **2 negative:** legacy main propagates connection refused, gateway startup propagates bind failure

Results:

- **Baseline: 27 passed, 6 failed (gateway crash, no schema, no replay) - preserved for gap analysis before hardening**
- **After hardening (try/except + REQUIRED_FIELDS + seen_nonces): 33 passed, 0 failed, both Rust and pure Python venvs**
- Large packet test updated: originally payload `{"device_id":..., "samples": "x"*5000}` missing required fields, after adding schema validation it failed, updated to include required fields + samples to still test buffer accumulation

How to Run the Full Set of Tests

```
# Setup
uv sync --frozen --all-groups
# or python3 -m venv /tmp/venv && pip install cryptography mlkem pytest

# Run all tests
uv run pytest -v
# or PYTHONPATH=. /tmp/venv/bin/python -m pytest -v
# Expected: 33 passed

# PQC round-trip
uv run python src/pqc_utils.py
# Expected: Encrypted packet len 845, metrics encaps_ms 0.06/0.82, Round-trip OK, rust_binding True/False

# 3-tier E2E (3 terminals)
# Terminal1 Cloud
uv run python src/cloud_service.py
# Terminal2 Gateway
uv run python src/edge_gateway.py
# Terminal3 Legacy
uv run python src/legacy_device.py
# Verify via curl health endpoints and sqlite3

# Docker
docker build -t edge-gateway:test .
docker build -t cloud-service:test .
docker-compose up --build
```

9.2 Manual Tamper Test (from 01_)

```
uv run python -c "
import legacy_device, edge_gateway
payload = legacy_device.generate_telemetry()
packet = bytearray(legacy_device.encrypt_payload(payload, legacy_device.PRE_SHARED_KEY))
packet[15] ^= 0xFF  # Corrupt one byte of ciphertext
try:
    edge_gateway.decrypt_payload(bytes(packet), legacy_device.PRE_SHARED_KEY)
except Exception as e:
    print('Tamper detected successfully:', type(e).__name__)
"
# Expected: Tamper detected successfully: InvalidTag
```

9.3 Packet Overhead Measurement (from 01_)

```
uv run python -c "
import json, legacy_device
raw = json.dumps(legacy_device.generate_telemetry()).encode('utf-8')
enc = legacy_device.encrypt_payload(legacy_device.generate_telemetry(), legacy_device.PRE_SHARED_KEY)
print(f'Plaintext Size:  {len(raw)} bytes')
print(f'Wire Packet:     {len(enc)} bytes (Nonce: 12, Tag: 16, Overhead: 28 bytes)')
"
# Plaintext ~95B, Wire ~123B, Overhead 28B constant
```

9.4 PQC Round-Trip (from 03_ + 05_)

```
uv run python src/pqc_utils.py
# Testing PQC utils round-trip (rust_binding=True/False)...
# Encrypted packet len 845, metrics {'encaps_ms': 0.06, 'aes_encrypt_ms': 1.73, 'total_ms': 1.82, 'ct_len': 768, ...}
# Decrypted: b'{"device_id":"sensor-node-01","temperature":21.8}', metrics {'decaps_ms': 0.07, ...}
# Round-trip OK!
```

9.5 E2E Test (from 05_ + 06_)

```
# 2 legacy sends → DB 2 rows, cloud health db_rows 2, avg_temp 21.8, decaps_ok 2, success_rate 1.0
# Gateway health received 2, forwarded 2
# Tested via: uv run python src/cloud_service.py + edge_gateway.py + legacy_device.py x2
# Verified via curl health endpoints + sqlite3
```

E2E Test Evidence (Last tested 2026-10-11):

```
# Terminal1 Cloud
uv run python src/cloud_service.py  # generates cloud_public_key.bin 800B + telemetry.db
# Expected: Cloud Service listening for gateway on port 6000... Cloud health/analytics listening on 6001...

# Terminal2 Gateway
uv run python src/edge_gateway.py  # listens 5000 + 5001, reads cloud_public_key.bin if exists
# Expected: Edge Gateway listening for legacy devices on port 5000...

# Terminal3 Legacy x2
uv run python src/legacy_device.py
uv run python src/legacy_device.py

# Verify E2E
curl http://127.0.0.1:5001/health  # {"received":2,"forwarded_to_cloud":2,"fallback_local":0}
curl http://127.0.0.1:6001/health  # {"db_rows":2,"analytics":{"avg_temperature":21.8,"avg_humidity":55.4},"pqc":{"decaps_ok":2,"success_rate":1.0,"avg_decaps_ms":0.04}}
sqlite3 telemetry.db "SELECT COUNT(*), AVG(temperature) FROM telemetry;"  # 2|21.8
```

Fallback evidence: stop cloud, send legacy → `gateway_fallback.log` has entry like `1726478400 {"device_id":"sensor-node-01",...}`, health 5001 shows `fallback_local=1`, `cloud_fail=1`.

CI evidence: `origin/main` run `38103152286` and onwards with success, passed 33/33 tests.

9.6 Fallback Test (from 04_ + 05_)

```
# Stop cloud, keep gateway running, send legacy
# Check gateway_fallback.log → has <timestamp> <json>
# Health 5001 shows fallback_local++ and cloud_fail++
# Expected: fallback log entry + metrics, no data loss (business continuity)
```

9.7 CI Evidence

- `origin/main` run `38103152286` success 32s after fixes (os import + PYTHONPATH + dual-wheel)
- `arena` run `38102098201` success 38s
- Both run `pytest -v` 33 tests, `pqc_utils.py` round-trip, import checks

---

10. Final Evaluation (full details from 06_final_evaluation.md v3)

10.1 Overview

We modernized from initial 2-tier baseline (legacy sensor → gateway, 27/33 tests) into 3-tier system with PQC (legacy → gateway → cloud) that works end-to-end and passes **33/33 tests**. Gateway now validates schema, checks replay, stays alive on malformed input, forwards via ML-KEM-512 hybrid to cloud, with fallback to local log. Cloud stores telemetry in SQLite and exposes analytics via health endpoints. CI/CD and Docker respect `pyproject.toml` + `uv.lock` + `.python-version`.

E2E evidence: 2 legacy sends → gateway `received 2, forwarded 2`, cloud `db_rows 2, avg_temp 21.8, decaps_ok 2, success_rate 1.0`, `telemetry.db` has 2 rows. Fallback evidence: cloud down → `gateway_fallback.log` has entry, `fallback_local=1`.

10.2 Strengths (with evidence)

- **End-to-end 3-tier works:** Legacy AES-128-GCM PSK (TCP 5000, EOF) → gateway decrypt + validate + replay check → PQC hybrid `ct(768)||nonce(12)||aes_ct` ~845B → cloud decaps + SQLite insert. Satisfies core requirement: integrate ML-KEM into at least one path (Link2 internet-facing) while preserving Link1 (no firmware change for 2KB SRAM sensor).
- **Robust gateway (was crashing):** Previously gateway crashed on `InvalidTag` exit 1. Now wrapped in `try/except InvalidTag, ValueError, UnicodeDecodeError, JSONDecodeError`, logs warning, continues. Schema validation `REQUIRED_FIELDS` + replay cache `seen_nonces=set()` stores 12B random nonces seen → 33/33 (was 27/33).
- **Minimal but functional PQC:** Library `mlkem==0.0.3` supports both Rust binding (rust.so 909KB, fast) and pure Python (fastmath.so 80KB, empty **init**.py) wheels via dual-wheel fix with `RUST_BINDING` flag, chosen over `kyber-py` and `liboqs` for simplicity. ML-KEM-512 selected for lowest overhead 800B pk + 768B ct = 1568B vs 2272B for 768. Hybrid with AES-256-GCM ss 32B as key. Measured encaps ~0.03ms Rust / 0.82ms pure Python, decaps ~0.04ms / 1.46ms negligible for gateway.
- **Simple DB as observability:** SQLite `telemetry.db` stdlib, 1 table, no ORM, `COUNT, AVG(temp/humidity), last 10`, exposed via cloud health 6001 JSON and gateway health 5001 metrics - managers see PQC vs fallback, avg temp, recent telemetry.
- **Fallback ensures availability:** If `cloud_public_key.bin` missing or cloud unreachable (TCP 6000 refused/timeout), gateway appends `<timestamp> <json>` to `gateway_fallback.log` + `fallback_local++`. Cloud decaps fail → `decaps_fail++` but stays alive. Business continuity per CISA hybrid guidance.
- **Operations complete but simple:** CI via `astral-sh/setup-uv` reading `.python-version` + `uv.lock`, `uv sync --frozen --all-groups` + `pytest -v`; Dockerfile `python:3.13-slim` + `uv sync --frozen --no-install-project` (respects lock); docker-compose with named volume `pqc-shared` for pk sharing, healthchecks, `depends_on healthy`; logging to files (`gateway.log`, `cloud.log`, `pqc.log`) to keep stdout clean for tests; health endpoints via stdlib `http.server` (no Flask).

10.3 Weaknesses and Technical Debt (combined, with why it matters, full from 06_)

Weakness / Debt	Why It Matters (plain English)	Impact	Future Mitigation
Global 16B PSK (Pre-Shared Key) for all devices `legacy_secret_16`	All sensors share same 16B key. If one device physically stolen and attacker reads its Flash ROM chip, attacker can decrypt all Link1 traffic and forge fake temperature data for all devices.	High if device tampered	Per-device subkeys derived from master key, stored in protected Flash, key rotation
Replay cache no TTL (Time To Live), no persistence	`seen_nonces=set()` is Python set in RAM that remembers 12B nonces seen to block replay. Two problems: (1) grows forever → could OOM (Out Of Memory) with many devices, (2) **On restart, set cleared → replayed packets accepted again**: when gateway process restarts, RAM wiped, set empty `set()`, so attacker can replay old captured packet and gateway thinks it is new because nonce not in empty set.	Medium (memory leak, replay after restart)	LRU (Least Recently Used) cache with TTL e.g., 5 min, or timestamp window (only accept packets whose timestamp is within last 5 min), persist cache to file/DB
File-based pk (public key) distribution, no auth (authentication)	`cloud_public_key.bin` 800B has no signature. If attacker replaces file on disk, they can do MITM (Man-In-The-Middle) on Link2 gateway→cloud.	Medium (MITM)	Sign pk file with private key, use certificate, or fetch via TLS with pinning, or KMS (Key Management System)
No key rotation	ML-KEM dk (decapsulation key = private key 1632B) stays in memory forever, never changed. If leaked, all past and future Link2 messages compromised (no forward secrecy = old messages can be decrypted).	Medium (long-term exposure)	Rotate daily, store dk in HSM (Hardware Security Module) or encrypted file, versioned keys
No TLS (Transport Layer Security) on Link2	PQC KEM gives confidentiality but no server authentication beyond file-based pk. Gateway does not know it really talks to real cloud.	Medium (no server auth)	Add TLS 1.3 with PQC certificates or hybrid certs

Weakness / Debt	Why It Matters (plain English)	Impact	Future Mitigation
Single-threaded gateway (data plane) - concurrent starvation risk	**2 threads total:** (1) main TCP data thread `accept()` + blocking `recv()` = waits for one sensor at a time, (2) health HTTP thread 5001. **Concurrent starvation** = while gateway processes one sensor (decrypt, validate, forward to cloud), other sensors that connect at same time wait in TCP queue (they starve, not served). If 100 sensors send at once, they timeout. This does NOT contradict single-threaded point - we intentionally said single-threaded for simplicity, 2 threads total but only 1 handles sensor data.	Low (scalability)	`asyncio` (asynchronous I/O) or thread pool (rejected for simplicity)
Logs no rotation, DB no backup	`gateway.log`, `cloud.log`, `pqc.log` grow forever → disk full. `telemetry.db` single file, no backup → data loss if disk fails.	Low	`RotatingFileHandler`, periodic DB backup/replication

We intentionally kept these for simplicity per course guidance: keep it plain and simple, don't make project bigger than needed.

10.4 Risks (Consolidated R1-R7, full)

#	Risk	Severity	Status	Remaining (what still needs fixing)
R1	Gateway DoS (Denial of Service) via malformed input (crash on InvalidTag)	High	**Mitigated:** try/except + schema + replay, now 33/33 pass, stays alive	Large payload could still cause high CPU when decrypting, should add rate limiting
R2	Replay attacks (stale telemetry reinjected)	Medium	**Mitigated:** seen_nonces set rejects replayed nonce (if nonce already seen, drop)	No TTL (Time To Live), restart clears cache → replay possible after restart. Should add timestamp window: only accept packets where `now - timestamp < 5 min`, and keep nonces only for that window
R3	PSK (Pre-Shared Key) extraction (physical tampering)	Medium	**Not mitigated:** global PSK still	Per-device keys, protected Flash
R4	PQC performance & memory overhead	Medium	**Mitigated/measured:** encaps 0.03ms Rust / 0.82ms pure Python, decaps 0.04ms / 1.46ms negligible for gateway (Linux/Pi)	800B pk (public key) + 768B ct (ciphertext) overhead matters for constrained radio. **Constrained radio** = low-power wireless like LoRaWAN, Zigbee, BLE, NB-IoT with small MTU (e.g., 51-250B max), low bandwidth, battery-powered. Sending 1568B extra each message is heavy for them (needs fragmentation, more battery). In our simulated TCP/Ethernet setup it's fine, but for real battery sensor with LoRa it would be problem.
R5	Scalability & concurrent starvation	Low	**Not mitigated:** single-threaded data plane (2 threads total, 1 for data)	asyncio or thread pool future
R6	File-based pk (public key) MITM (Man-In-The-Middle)	Medium	**Not mitigated:** pk file no auth (authentication)	Sign pk or use secure distribution
R7	DB (Database) no backup	Low	**Not mitigated:** single file `telemetry.db`	Periodic backup

Overall, high-severity R1 fixed, medium R2 partially fixed, others documented as debt.

10.5 What If ML-KEM Not Fully Integrated? (Per Course Q&A: OK to Document)

Current integration minimal viable and works E2E (tested). If PQC path fails:

- **Cloud down or pk (public key) missing:** Gateway fallback to `gateway_fallback.log` ensures no data loss (business continuity). Operator can replay log later.
- **Decaps fail (decapsulation = private key decrypt fails):** Cloud increments `decaps_fail`, logs warning, stays alive, `success_rate` exposed via health 6001 shows degradation.
- **Gaps documented:** No key rotation, no cert (certificate), ML-KEM-512 only (not 768 for long-term confidentiality), file-based distribution. For peer review, sufficient - 3-tier works, PQC path exists, fallback documented, per course guidance that incomplete integration is OK if documented in final evaluation.

10.6 Conclusion

System meets peer review requirements with simple design: legacy compatibility preserved (Link1 unchanged, no firmware update needed for 2KB SRAM device), PQC added on internet-facing Link2 via gateway offload pattern, with SQLite DB and analytics as observability and fallback for availability. Tests improved from 27/33 to 33/33 after hardening. CI/CD and Docker respect `pyproject.toml` + `uv.lock` + `.python-version` for reproducibility.

Remaining work is to clear technical debt (global PSK, replay cache TTL, pk authentication, key rotation, TLS, scalability) prioritized for future sprints. LLM was used as an untrusted assistant (10 interactions logged in 02_design_decisions_n_key_llm_interactions.md) and manually evaluated via pytest. Insecure or wrong suggestions (static nonce, `MLKEM512.keygen()`, Flask, requirements.txt) were rejected or corrected.

---

11. References

- NIST FIPS 203 ML-KEM standardised in August 2024 - https://www.encryptionconsulting.com/education-center/ml-kem-fips-203/
- ML-KEM sizes: 800B pk (public key / encapsulation key), 768B ct (ciphertext) - https://vucense.com/privacy-sovereignty/post-quantum-cryptography/ml-kem-fips-203-implementation-guide-2026/
- EU roadmap, hybrid for legacy, hardware replacement may be needed - https://www.helpnetsecurity.com/2025/07/10/david-warburton-f5-labs-eu-pqc-roadmap/
- legacy constraints, computational overhead - https://eureka.patsnap.com/report-how-to-adapt-post-quantum-standards-to-existing-legacy-systems
- CISA PQC initiative, phased migration - https://www.cisa.gov/topics/risk-management/quantum

---

12. Appendix

12.1 Files (full codebase)

- `src/legacy_device.py` (AES-128-GCM, 12B nonce, EOF)
- `src/edge_gateway.py` (hardened, PQC forwarding, fallback, health 5001, file logging, metrics, replay cache)
- `src/pqc_utils.py` (dual-wheel Rust/pure Python support, constants, keygen, encrypt/decrypt, metrics, logging)
- `src/cloud_service.py` (TCP 6000 data, health 6001 analytics, SQLite DB, init_db, insert_telemetry, get_analytics, metrics)
- `tests/unit/test_legacy_device_and_gateway.py` (5 tests), `tests/unit/test_security_and_invalid_inputs.py` (11 tests), `tests/integration/test_gateway_tcp_flow.py` (2 tests), `tests/integration/test_gateway_failure_and_robustness.py` (10 tests, including large packet >1024B), `tests/conftest.py` (PROJECT_ROOT + SOURCE_DIRECTORY in sys.path, GatewayProcess with queue.Queue, ephemeral ports)
- `documentation/01_baseline_analysis.md` (full baseline), `02_design_decisions_n_key_llm_interactions.md`, `03_mlkem_integration.md`, `04_migration_fallback_strategy.md`, `05_operations.md`, `06_final_evaluation.md`, `part1_documentation.md` (this file consolidating our other documentation in parts so reviewers can refer only to this file)
- `documentation/archive/02_rough_document_legacy_device.md`, `03_how_to_run_baselinecode_without_cloud.md`, `04_understand_links_between_docker_files.md` (archive, not required but kept)
- `.github/workflows/ci.yml` (test + docker-build, setup-uv, PYTHONPATH, --all-groups), `Dockerfile` (python:3.13-slim, uv sync --frozen, HEALTHCHECK), `docker-compose.yml` (cloud+gateway, healthchecks, depends_on healthy, volume pqc-shared)
- `pyproject.toml`, `uv.lock`, `.python-version` (3.13), `README.md`

12.2 LLM Interactions Full Log (summary, full in 02_)

See 02_design_decisions_n_key_llm_interactions.md sections 1.1-1.10 for our major LLM prompt interactions and human review (accepted/modified/rejected).

Key prompts (10 LLM Interactions + 3 human fixes):

- Interaction1: "Write two Python scripts using cryptography library... legacy_device.py encrypts... edge_gateway.py listens..."
- Interaction2: "Should we use WebSockets or HTTP REST..."
- Interaction3: "How should edge gateway know when sensor finished sending..."
- Interaction4: "I am new to this codebase. Please analyze src/legacy_device.py and src/edge_gateway.py... propose test approach..."
- Interaction5: "Please proceed to generate code for topics 1 to 4 only..."
- Interaction6: "My edge_gateway.py crashes when it receives malformed AES-GCM packets... How should I harden it to pass these tests...?"
- Interaction7: "I need to integrate ML-KEM-512 into my edge-cloud system. I have mlkem==0.0.3 library. Show how to generate keypair, encapsulate..."
- Interaction8: "Create a cloud_service.py that listens on TCP 6000 for PQC packets... stores in SQLite... health/analytics endpoint on 6001..."
- Interaction9: "Extend edge_gateway.py to forward telemetry to cloud via PQC: try load cloud_public_key.bin, if missing or cloud unreachable, append JSON to gateway_fallback.log..."
- Interaction10: "Generate GitHub Actions CI and Dockerfile and docker-compose for Python project using uv package manager, with pyproject.toml, uv.lock, .python-version 3.13..."
- Fix11: "ModuleNotFoundError: No module named 'src'" → robust sys.path handling in conftest + edge_gateway + cloud_service
- Fix12: "ImportError: cannot import name 'ML_KEM' from 'mlkem'" → dual-wheel support Rust vs pure Python
- Fix13: "NameError: name 'os' is not defined" → add import os in cloud_service